Generative AI, DV2658

LAB 1: Variational Autoencoder

Amir Yavariabdi

Blekinge Institute of Technology

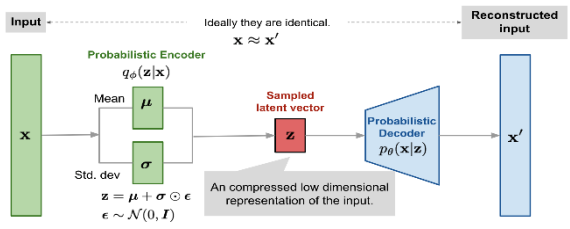
Contact: aya@bth.se

Date: November 06, 2026

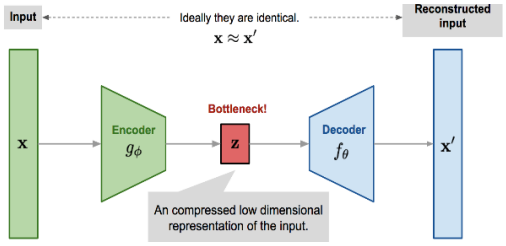
Exercise 1: Time-Series Anomaly Detection using LSTM-VAE

Objective:

The objective of this lab is to implement an LSTM Variational Autoencoder (VAE). Unlike standard Autoencoders that map input to a fixed vector, a VAE maps input to a probability distribution (Gaussian). This implementation specifically uses Long Short-Term Memory (LSTM) layers to capture temporal dependencies in multi-dimensional sensor data.



(A): Variational Autoencoder



(B): Autoencoder

Key Concepts of VAE:

- Encoder: Compresses a window of time-series data into a mean (μ) and variance (σ).
- Latent Space: A 100-dimensional space representing the underlying rules of the signal.
- Reparameterization Trick: A method to allow backpropagation through random sampling by using $z = \mu + \sigma \cdot \epsilon$.
- Decoder: Reconstructs the original signal from the latent vector.

Steps:

1. **Download the required files:**
   - Go to the course page on Canvas. Locate the folder data_lab1.
   - Download the entire folder to your local machine.
   - Extract the contents if the folder is compressed.
   - Inside the folder, find the file: **sample_data.dat**
   - Make sure this file is accessible from your working directory (e.g., the same folder as your notebook or script).

2. Importing Necessary Libraries

To build a deep learning model for time-series, we need tools for numerical processing, neural network construction, and backend mathematical operations.

- TensorFlow/Keras: The main framework used to construct and train neural networks, including LSTM and Dense layers.
- Keras Backend (K): Provides low-level tensor operations needed for:
    - Implementing the reparameterization trick (sampling from a latent distribution)
    - Defining custom loss functions for the VAE
- NumPy: For numerical computations
- Pandas: For loading and handling the dataset
- Matplotlib: For visualization of results

Run the following code to import all necessary libraries:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import tensorflow as tf
from tensorflow.keras import backend as K
from tensorflow.keras.layers import Input, LSTM, RepeatVector, Dense, Lambda
from tensorflow.keras import layers, models
from tensorflow.keras.models import Model
from tensorflow.keras import losses
```

3. Data Preprocessing and Windowing

In this step, we prepare the time-series data for input into the Variational Autoencoder (VAE).

The dataset (sample_data.dat) contains:

- 419 observations (time steps)
- 13 sensor features
- Values already **normalized between 0 and 1**, which is crucial to:
    - Prevent gradient explosion
    - Ensure stable training of the VAE (especially the KL-divergence term)

Windowed Data Sample

Raw time-series data is typically stored as a 2D array: *(Time steps, Features)* → *(419, 13)*.

However, an LSTM expects 3D input: *(Samples, Timesteps, Features)*.

To achieve this, we use a sliding window approach. Then, we group the data into overlapping sequences of 3 timesteps:

- Sample 1: Contains rows [0, 1, 2]
- Sample 2: Contains rows [1, 2, 3] (The window slides forward by one step).
- Sample 3: Contains rows [2, 3, 4] (The window slides forward by one step).

This creates:

- 415 total samples
- Each sample has shape *(3, 13)*

Final dataset shape: *(415, 3, 13)*

```python
def get_raw_data():
    # 1. Load the binary data and reshape to 13 columns
    data = np.fromfile('sample_data.dat').reshape(419, 13)
    return data

def get_windowed_data(data, timesteps=3):
    # 2. Apply the sliding window logic
    dataX = []
    for i in range(len(data) - timesteps - 1):
        x_window = data[i:(i + timesteps), :]
        dataX.append(x_window)
    return np.array(dataX)


raw_data = get_raw_data()
windowed_data = get_windowed_data(raw_data)

# Create generic headers for the 13 features
headers = [f"Sensor_{i+1}" for i in range(13)]

# A. Show Table (First 5 Rows of the raw data)
print("--- FIRST 5 ROWS OF RAW DATA (13 FEATURES) ---")
df = pd.DataFrame(raw_data, columns=headers)
print(df.head(5))

# B. Show 5 Samples of Windowed Data (Each sample is 3 timesteps)
print("\n--- FIRST 5 WINDOWED SAMPLES (FOR LSTM INPUT) ---")
for i in range(5):
    print(f"\nSample {i+1} (Shape: {windowed_data[i].shape}):")
    # Displaying each sample as a small dataframe for clarity
    sample_df = pd.DataFrame(windowed_data[i], columns=headers)
    sample_df.index = [f"t-{2-j}" for j in range(3)] # Labeling time steps
    print(sample_df)
```

Details:

- **Reshaping**:

  The binary file is loaded and reshaped into **13 parallel sensor streams**, one per feature.

- **Sliding Window**

  By creating overlapping windows:
  - The model observes short temporal sequences
  - Sudden changes (e.g., from $t-2 \rightarrow t-0$) can be learned effectively

- **Normalization**

  Since values are between **0 and 1**:
  - Training becomes more stable
  - The **KL-divergence loss** in the VAE remains numerically well-behaved

4. Defining the Variational Autoencoder (VAE) Architecture

A **Variational Autoencoder (VAE)** differs from a standard autoencoder in a fundamental way: Instead of mapping an input to a single point in latent space, it maps it to a **probability distribution**.

Architecture Overview

The model consists of three main components:

1. **Encoder** → Compresses input into a latent distribution
2. **Latent Space (Variational Layer)** → Samples from the distribution
3. **Decoder** → Reconstructs the original sequence

Model Summary

The architecture you will implement is shown below:

- Input: **(3 timesteps, 13 features)**
- Latent space: **100 dimensions**
- Output: **(3 timesteps, 13 features)**

Layer (type)	Output Shape	Param #	Connected to
input_layer_29 (InputLayer)	(None, 3, 13)	0	-
lstm_15 (LSTM)	(None, 32)	5,888	input_layer_29[0][0]
dense_34 (Dense)	(None, 100)	3,300	lstm_15[0][0]
dense_35 (Dense)	(None, 100)	3,300	lstm_15[0][0]
vae_layer_2 (VAELayer)	(None, 100)	0	dense_34[0][0], dense_35[0][0]
repeat_vector_2 (RepeatVector)	(None, 3, 100)	0	vae_layer_2[0][0]
lstm_16 (LSTM)	(None, 3, 32)	17,024	repeat_vector_2[0][0]
lstm_17 (LSTM)	(None, 3, 13)	2,392	lstm_16[0][0]

The Encoder

The Encoder takes the (3, 13) input and compresses it.

- LSTM Layer: Captures temporal dependencies across the 3 timesteps.
  - *Output: (None, 32)*
- Latent Parameters: Two separate Dense layers that represent the center and spread of our data in the latent space.
  - z_mean: Represents the center of the distribution
  - z_log_sigma: Represents the spread (log variance)

The Reparameterization Trick

We cannot backpropagate through a random sample. To fix this, we use the formula:

$$z = \mu + e^{(0.5 \cdot \log \sigma^2)} \cdot \epsilon$$

where $\epsilon \sim N(0,1)$ is a random noise. This reformulation keeps the model fully differentiable.

The Decoder

The Decoder takes the sampled latent vector and reconstructs the original *(3, 13)* shape.

- RepeatVector: Replicates the latent vector 3 times to recreate the time dimension.
- LSTM Decoder: Reconstructs the sequence of 13 features.

```
class VAELayer(layers.Layer):
    def __init__(self, **kwargs):
        super(VAELayer, self).__init__(**kwargs)

    def call(self, inputs):
        z_mean, z_log_sigma = inputs
        # Calculate KL loss internally to avoid tensor conversion errors
        kl_loss = -0.5 * K.mean(1 + z_log_sigma - K.square(z_mean) - K.exp(
z_log_sigma))
        self.add_loss(kl_loss)

        # Reparameterization trick
        epsilon = tf.random.normal(shape=(tf.shape(z_mean)[0], latent_dim))
        return z_mean + K.exp(z_log_sigma / 2) * epsilon

# Parameters from example.py
intermediate_dim = 32
latent_dim = 100
batch_size = 1

# --- ENCODER ---
inputs = layers.Input(shape=(timesteps, input_dim,))
h = layers.LSTM(intermediate_dim)(inputs)
z_mean = layers.Dense(latent_dim)(h)
z_log_sigma = layers.Dense(latent_dim)(h)

# Use the custom layer
z = VAELayer()([z_mean, z_log_sigma])

# --- DECODER ---
decoder_h = layers.LSTM(intermediate_dim, return_sequences=True)
decoder_mean = layers.LSTM(input_dim, return_sequences=True)

h_decoded = layers.RepeatVector(timesteps)(z)
h_decoded = decoder_h(h_decoded)
x_decoded_mean = decoder_mean(h_decoded)

# --- MODELS ---
vae = models.Model(inputs, x_decoded_mean)

# The loss function now only needs to handle MSE (KL is added by VAELayer)
vae.compile(optimizer='adam', loss='mse')

# For Inference: Encoder and Generator
encoder = models.Model(inputs, z_mean)

decoder_input = layers.Input(shape=(latent_dim,))
_h_decoded = layers.RepeatVector(timesteps)(decoder_input)
_h_decoded = decoder_h(_h_decoded)
```

```
_x_decoded_mean = decoder_mean(_h_decoded)
generator = models.Model(decoder_input, _x_decoded_mean)

print("Model built and compiled successfully.")
```

Details:

- The custom layer defines the "Variational" bridge between the Encoder and Decoder. It handles the probabilistic sampling and the regularization loss.
    - KL Divergence (kl_loss): This formula calculates how much the learned distribution deviates from a standard Normal distribution (Mean=0, Std=1). By adding this as a loss, we force the latent space to be continuous and compact.
    - Reparameterization Trick: In a VAE, we need to sample a point z from the distribution defined by z_mean and z_log_sigma. However, backpropagation cannot flow through a random sampling process. We solve this by adding a random constant epsilon, making the process $z = \mu + \sigma \cdot \epsilon$. This allows the model to remain differentiable and trainable.
- The encoder condenses a sequence of sensor readings into a simplified summary representation.
    - LSTM Layer: This layer processes the 3 timesteps. Because it is an LSTM, it captures the temporal trend (e.g., is Sensor 1 increasing or decreasing over these 3 steps?).
    - Mean and Log-Sigma: Instead of outputting one vector, the encoder outputs two: one representing the most likely state (z_mean) and one representing the uncertainty or spread (z_log_sigma).
- The decoder takes the abstract 100-dimensional latent vector and reconstruct the original 3×13 sequence.
    - RepeatVector: The latent vector z is a flat vector (1D). To turn it back into a time-series, we must repeat it for the number of timesteps (3).
    - LSTM Decoder: These layers learn how to turn the summary back into a realistic sequence of 13 features. We use return_sequences=True because we want the output for every timestep, not just the final one.
- Model compilation finalizes the model architecture and defines the learning strategy.
    - Loss Function (MSE): While the KL loss is handled inside the VAELayer, we use Mean Squared Error (MSE) here to ensure the reconstructed output is as close to the original input as possible.
    - Optimizer (Adam): A popular gradient descent algorithm that automatically adjusts the learning rate for faster convergence.
- Inference models (encoder and generator) create standalone models for specific tasks after training.
    - Encoder Model: Used to transform data into latent vectors for visualization or clustering.
    - Generator Model: Used to create "synthetic" sensor data by feeding a random latent vector into the decoder. This is useful for testing what the model considers "normal" data.

5. Training and Loss Functions

Once the VAE architecture is defined, we train the model to learn how to reconstruct the input time-se ries data while organizing the latent space.

Training the Model

The VAE is trained by feeding the same data as both input and target:

```
vae.fit(x, x, epochs=20, batch_size=batch_size)
```

- x: Windowed time-series data of shape **(415, 3, 13)**
- The model learns to reconstruct the input sequence from its compressed latent representation

Loss Functions in a VAE

A Variational Autoencoder uses **two complementary loss components**:

1. Reconstruction Loss (Mean Squared Error - MSE)

This measures how well the output matches the input.

$$MSE = \frac{1}{N}\sum(x - \hat{x})^2$$

- Encourages accurate reconstruction
- Penalizes large differences between:
  - Original input (x)
  - Reconstructed output ($\hat{x}$)

2. KL Divergence Loss

This regularizes the latent space by pushing it toward a **standard normal distribution**:

$$D_{KL}\big(q(z|x) \parallel\ p(z)\big) = -\frac{1}{2}\sum(1\ +\ log\ \sigma^2\ -\mu^2 - \sigma^2\ )$$

- Ensures:
  - Smooth latent space
  - Meaningful interpolation between points
- Prevents the model from memorizing data

Combined Objective

The total loss is:

$$Total\ Loss = Reconstruction\ Loss + KL\ Divergence$$

- **MSE** → makes outputs accurate
- **KL Divergence** → makes latent space structured

Note: In this implementation, the **KL loss is automatically added inside the custom VAELayer**, so we only specify 'mse' during compilation.

What Happens During Training?

At each epoch:

1. The encoder compresses input sequences
2. The latent layer samples a representation
3. The decoder reconstructs the sequence
4. The model updates weights to:
   - Improve reconstruction
   - Regularize the latent space

Practical Insight

- If **MSE is low but KL is high** → good reconstruction, poor latent structure
- If **KL is low but MSE is high** → structured latent space, poor reconstruction
- A good VAE balances both

6. Evaluation and Anomaly Detection

After training the VAE, we use it to reconstruct the input data and identify anomalies based on reconstruction quality.

Core Idea: Reconstruction Error

The key concept behind anomaly detection with a VAE is:

- **Normal patterns are reconstructed well → Low error**
- **Abnormal patterns are reconstructed poorly → High error**

Reconstruction Error (MSE)

We measure the difference between the input x and reconstruction $\hat{x}$ using Mean Squared Error:

$$\text{MSE} = \frac{1}{N}\sum(x - \hat{x})^2$$

- Low MSE → Normal behavior
- High MSE → anomaly

Detection Mechanism

- The VAE learns **normal temporal patterns** during training
- When it encounters **unseen or abnormal patterns**:
  - It tries to reconstruct them as *normal*
  - This creates a **large reconstruction error**

Thresholding Strategy

We use **percentile-based thresholding**:

- Define anomalies as the **top 10% highest errors**
- Threshold = 90th percentile of MSE values

```
# 1. Generate predictions for the whole dataset
preds = vae.predict(x, batch_size=batch_size)

# 2. Calculate Reconstruction Error (MSE)
# We average the squared difference across timesteps and features
mse_error = np.mean(np.power(x - preds, 2), axis=(1, 2))

# 3. Define the Anomaly Threshold
# Identifying the top 10% highest errors as potential anomalies
threshold = np.percentile(mse_error, 90)
is_anomaly = mse_error > threshold

print(f"Threshold set at: {threshold:.4f}")
print(f"Anomalies detected: {np.sum(is_anomaly)} sequences")

# 4. Plotting the results
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 10), sharex=True)
```

```
# Plot A: Signal Reconstruction (Focusing on Feature index 3)
feature_idx = 3
ax1.plot(x[:, 0, feature_idx], label='Original Data', color='blue', alpha=0
.5)
ax1.plot(preds[:, 0, feature_idx], label='VAE Reconstruction', color='orang
e', linestyle='--')

# Highlight anomalous points with red dots
anomaly_indices = np.where(is_anomaly)[0]
ax1.scatter(anomaly_indices, x[anomaly_indices, 0, feature_idx],
            color='red', label='Anomalies', s=25, zorder=5)
ax1.set_title(f"Signal Reconstruction and Anomalies (Feature {feature_idx})
")
ax1.legend()

# Plot B: Reconstruction Error Level
ax2.plot(mse_error, color='black', label='MSE Error')
ax2.axhline(y=threshold, color='red', linestyle='--', label='Anomaly Thresh
old')

# Corrected: .fill_between() without the 'set_' prefix
ax2.fill_between(range(len(mse_error)), 0, mse_error, where=is_anomaly, col
or='red', alpha=0.3)

ax2.set_title("Reconstruction Error (Anomaly Score)")
ax2.set_xlabel("Sample Index")
ax2.legend()

plt.tight_layout()
plt.show()
```

Conclusion of Exercise 1:

In this exercise, we implemented a Long Short-Term Memory Variational Autoencoder (LSTM-VAE) to identify irregularities in multi-dimensional sensor data. By using the temporal learning capabilities of LSTM layers and the probabilistic nature of Variational Autoencoders, we moved beyond simple thresholding to a model that understands the normal behavioral context of a system.

Key Points:

- Temporal Context: The sliding window approach (3 timesteps) allowed the model to learn not just the values of the sensors, but how they transition over time.
- Latent Distribution: By forcing the latent space into a Gaussian distribution via KL-Divergence, the model learned a robust, generalized representation of the "normal" state.
- Reconstruction Error as a Metric: We utilized the Mean Squared Error (MSE) between the input and output as an "Anomaly Score." High scores indicated patterns that the model had never seen during training, successfully flagging them as anomalies.

To enhance the performance of both the VAE architecture and the anomaly detection strategy, consider the following advancements:

1. **Improving the VAE Architecture**

- Bidirectional LSTMs: Instead of standard LSTMs, use Bidirectional layers in the encoder. This allows the model to process the sequence in both forward and backward directions, capturing a fuller context of the temporal patterns.
- Deepening the Hidden Layers: Increase the number of intermediate_dim layers or add Dropout layers to prevent overfitting and improve the latent space's ability to represent complex non-linear relationships.
- Beta-VAE: Introduce a β hyperparameter to the KL-loss term. Increasing $\beta > 1$ can lead to more "disentangled" latent representations, making it easier to identify which specific latent feature corresponds to an anomaly.

2. **Enhancing Anomaly Detection Strategy**

- Dynamic Thresholding: Instead of a static 90th percentile threshold, implement a Moving Window Threshold (e.g., based on the rolling mean and standard deviation of the error). This helps the model adapt to seasonal changes or expected long-term drift in the data.
- Feature-Wise Error Analysis: Instead of averaging MSE across all 13 sensors, monitor the error for each sensor individually. This allows you to identify which specific sensor caused the anomaly, providing better diagnostic information.
- Probabilistic Anomaly Scores: Instead of using raw MSE, calculate the Mahalanobis Distance or the Log-Likelihood of the reconstruction. Since the VAE is probabilistic, these metrics can provide a more statistically sound confidence interval for what constitutes an "unlikely" event.
- Multi-Scale Windows: Train the model using multiple window sizes (e.g., 3 steps, 10 steps, and 30 steps) to detect both "point anomalies" (sudden spikes) and "collective anomalies" (slow-building failures).

Exercise 2: Generating Sentences from a Continuous Space

Objective:

In this exercise, you will implement a **Sequence-to-Sequence Variational Autoencoder (Seq2Seq VAE)** for natural language, following the ideas of Bowman et al. (2016), *Generating Sentences from a Continuous Space*. Unlike standard language models that treat sentences only as discrete sequences of words, this model:

- Maps every sentence to a **continuous latent space**
- Learns a **probability distribution** over sentences
- Enables:
    - **Reconstruction**: encoding a sentence and decoding it again
    - **Interpolation** between two sentences
    - **Generation** of new sentences by sampling from the latent space

In Exercise 1 the VAE was used to detect anomalies through reconstruction error. In this exercise the focus is on **representation and generation**: we want a latent space that is smooth, meaningful and can be sampled to produce new, grammatical sentences.

Key Concepts

- **Tokenization & Embedding**
    - Raw text is split into tokens (words and punctuation)
    - Every token gets an integer id, and an **embedding layer** maps each id to a dense vector that is learned during training
- **Latent Space**
    - The encoder compresses a full sentence into the parameters of a Gaussian distribution: a mean μ and a variance σ^2
    - A latent vector z is sampled from this distribution and given to the decoder
- **Teacher Forcing**
    - During training, the decoder receives the **true previous word** instead of its own prediction
    - This stabilises and accelerates learning
- **Posterior Collapse** (the central difficulty of text VAEs)
    - An LSTM decoder is a strong language model: it can produce plausible sentences without looking at z
    - If nothing prevents it, the model sets $q(z|x)$ equal to the prior $N(0, I)$ (the KL term becomes 0) and the decoder ignores z
    - Then every z decodes to the same few sentences, and interpolation or sampling becomes meaningless
    - This exercise uses four remedies: **KL annealing**, **word dropout**, **free bits**, and feeding **z to the decoder at every step**
- **Latent Space Interpolation and Sampling**
    - Two sentences give two latent vectors, points on the line between them decode to intermediate sentences
    - Sampling $z \sim N(0, I)$ and decoding it generates new sentences

Steps:

1. **Download the required files:**

- Go to the course page on Canvas. Locate the folder **data_lab1**.
- Download the entire folder to your local machine and extract it if it is compressed.
- Inside the folder, find the file **fra.txt**.

- Make sure this file is accessible from your working directory (the notebook expects `data_lab1/fra.txt`).

`fra.txt` contains **parallel sentences**: each line holds an English sentence and its French translation, separated by a tab. We only use the English side. The file is sorted from short to long sentences, and many English sentences appear several times (once per French translation).

2. Importing Necessary Libraries

To build a generative model for natural language, we need text processing tools and deep learning components:

- **NumPy** and **Matplotlib**: numerical operations and plots
- **NLTK**: `word_tokenize` for splitting sentences into tokens, and `sentence_bleu` for evaluation
- **TensorFlow / Keras 3**: building and training the model (`Embedding`, `Bidirectional`, `LSTM`, `Dense`, `Concatenate`)
- **keras.ops** and **keras.random**: backend-independent tensor operations and random sampling, used for the reparameterization trick

```
import numpy as np
import matplotlib.pyplot as plt

import nltk
from nltk.tokenize import word_tokenize
from nltk.translate.bleu_score import sentence_bleu, SmoothingFunction

import tensorflow as tf
import keras
from keras import layers, ops
from keras.layers import Input, LSTM, Dense, Embedding, Bidirectional, Conc
atenate, Layer
from keras.models import Model

# word_tokenize needs the Punkt models (newer NLTK uses "punkt_tab")
for pkg in ("punkt", "punkt_tab"):
    try:
        nltk.download(pkg, quiet=True)
    except Exception:
        pass

SEED = 42
np.random.seed(SEED)
tf.random.set_seed(SEED)
print("TensorFlow", tf.__version__, "| Keras", keras.__version__)
```

Details:

- **word_tokenize** splits raw text into words and punctuation
  - Example: "don't go." → ["do", "n't", "go", "."]
  - It needs the NLTK *punkt* models, which the cell downloads if they are missing
- **Fixed random seeds** make results more reproducible
  - Training on a GPU can still give slightly different numbers between runs

3. Data Preparation

Neural networks cannot process raw text, so every sentence is turned into sequences of integers. The function `get_text_data` produces three arrays, one for each role in teacher forcing:

- **Encoder input**: the sentence that is compressed into z
- **Decoder input**: the same sentence shifted one step to the right, starting with the start token
- **Target**: the words the decoder must predict

```
PAD, START, UNK, END = "<pad>", "\t", "<unk>", "<end>"
SPECIAL_TOKENS = [PAD, START, UNK, END]          # ids 0, 1, 2, 3


def get_text_data(data_path, num_samples=20000, min_freq=2, dedupe=True):
    # Load English sentences from fra.txt and build integer tensors for tea
cher forcing.
    with open(data_path, "r", encoding="utf-8") as f:
        lines = f.read().lower().split("\n")

    sentences, seen = [], set()
    for line in lines[: min(num_samples, len(lines) - 1)]:
        if "\t" not in line:
            continue
        english = line.split("\t")[0].strip()
        if dedupe and english in seen:
            continue
        seen.add(english)
        tokens = word_tokenize(english) + [END]
        sentences.append(tokens)

    # Replace rare words by <unk>
    counts = {}
    for s in sentences:
        for t in s:
            counts[t] = counts.get(t, 0) + 1
    rare = {t for t, n in counts.items() if n < min_freq and t not in SPECI
AL_TOKENS}
    sentences = [[UNK if t in rare else t for t in s] for s in sentences]
    n_unk = sum(t == UNK for s in sentences for t in s)

    vocab = sorted({t for s in sentences for t in s} - set(SPECIAL_TOKENS))
    id2token = SPECIAL_TOKENS + vocab
    token2id = {t: i for i, t in enumerate(id2token)}
    max_len = max(len(s) for s in sentences)

    n = len(sentences)
    enc_in = np.zeros((n, max_len), dtype="int32")
    dec_in = np.zeros((n, max_len), dtype="int32")
    target = np.zeros((n, max_len), dtype="int32")
    for i, toks in enumerate(sentences):
        ids = [token2id[t] for t in toks]
        enc_in[i, : len(ids)] = ids
        target[i, : len(ids)] = ids
        dec_in[i, 0] = token2id[START]
        dec_in[i, 1 : len(ids)] = ids[:-1]
```

```
    n_tok = sum(len(s) for s in sentences)
    print("Number of sentences  :", n)
    print("Vocabulary size      :", len(id2token), f"({len(rare)} rare word
s mapped to <unk>)")
    print(f"<unk> share of tokens: {100 * n_unk / n_tok:.1f} %")
    print("Max sequence length  :", max_len)
    return dict(sentences=sentences, id2token=id2token, token2id=token2id,
                max_len=max_len, enc_in=enc_in, dec_in=dec_in, target=targe
t)
```

Details:

- **Loading and cleaning**
  - The file is read line by line, lowercased, and split at the tab to keep the English sentence
  - **Duplicates are removed**: the first 20,000 lines contain about 12,300 unique English sentences
  - Each sentence is tokenized and the end token `<end>` is appended
- **Rare words become <unk>**
  - Almost half of all words occur only once in the data; the model cannot learn anything useful about them
  - Every word seen fewer than `min_freq` times is replaced by `<unk>`; this affects only about 2 % of all tokens but makes the vocabulary much smaller
- **Index mapping**: `token2id` (word → number) and `id2token` (number → word)
- **Zero padding**: all sequences are padded with `<pad>` (id 0) to the length of the longest sentence
  - The embedding layer uses `mask_zero=True`, so padded positions are skipped by the LSTMs
  - Padded positions are also excluded from the loss and from all accuracy measures

Four ids are reserved for special tokens:

Id	Token	Role
0	`<pad>`	Padding; masked out of the LSTMs and the loss
1	`\t`	Start token: the first input of the decoder
2	`<unk>`	Rare words, and words hidden by word dropout during training
3	`<end>`	End of sentence: generation stops when it is produced

Example: teacher forcing for the sentence "I see."

After tokenization the sentence becomes ['i', 'see', '.', '<end>']. At every timestep the decoder receives the previous word and must predict the next one:

Timestep (t)	Encoder input	Decoder input	Target
0	'i'	'\t' (start token)	'i'
1	'see'	'i'	'see'
2	'.'	'see'	'.'
3	'<end>'	'.'	'<end>'
4	<pad>	<pad>	<pad> (ignored in the loss)

Why embeddings instead of one-hot vectors?

A one-hot vector has one entry per vocabulary word, with a single 1. Multiplying a one-hot vector by a weight matrix simply selects one row of that matrix, which is exactly what an `Embedding` layer does, but without building the large one-hot tensor (for 20,000 lines it would need close to 1 GB of memory). The embedding vectors are learned, so words used in similar contexts end up with similar vectors.

4. Parameter Configuration

The next cell defines the hyperparameters for the data, the model and the VAE regularisation.

```
# Data
DATA_PATH = "data_lab1/fra.txt"
NUM_SAMPLES = 20000
MIN_FREQ = 2

# Training
BATCH_SIZE = 64
EPOCHS = 40
LEARNING_RATE = 1e-3

# Model
EMBED_DIM = 128
HIDDEN_DIM = 256
LATENT_DIM = 32

# VAE regularisation (anti posterior-collapse)
WORD_DROPOUT = 0.25
KL_WARMUP_EPOCHS = 2
KL_ANNEAL_EPOCHS = 10
BETA_MAX = 1.0
FREE_BITS = 0.25          # 0.25 nats x 32 dims = 8 nats of KL that cost not
hing

# Sampling
NUM_SAMPLES_TO_GENERATE = 5
```

Parameter	Value	Meaning
`NUM_SAMPLES`	20000	Lines read from fra.txt (about 12,300 unique sentences of 1 to 8 tokens)
`MIN_FREQ`	2	Words seen fewer than 2 times become `<unk>`
`BATCH_SIZE`	64	Sentences per gradient update (about 190 updates per epoch)
`EPOCHS`	40	Passes over the data
`LEARNING_RATE`	0.001	Adam learning rate; gradients are also clipped to norm 1
`EMBED_DIM`	128	Size of a word embedding
`HIDDEN_DIM`	256	LSTM units; the encoder is bidirectional, so it produces 2 × 256 features
`LATENT_DIM`	32	Size of z. Larger values add KL cost without adding useful capacity for short sentences
`WORD_DROPOUT`	0.25	Share of decoder input words replaced by `<unk>` during training
`KL_WARMUP_EPOCHS`	2	Epochs with β = 0 (no KL penalty)

Parameter	Value	Meaning
KL_ANNEAL_EPOCHS	10	Epochs over which β rises linearly from 0 to BETA_MAX
BETA_MAX	1.0	Final weight of the KL term (1.0 gives the standard VAE objective)
FREE_BITS	0.25	KL (in nats) per latent dimension that is not penalised: 32 × 0.25 = 8 nats in total

Note:

- **Choosing the dataset size**: with only 3,000 lines (about 1,700 very short sentences) the model memorises its training data, and almost every "generated" sentence is a copy of a training sentence. 20,000 lines give enough variety for the model to learn general sentence patterns.

5. Loading the Data and Verifying the Tensors

```
data = get_text_data(DATA_PATH, num_samples=NUM_SAMPLES, min_freq=MIN_FREQ)

enc_in, dec_in, target = data["enc_in"], data["dec_in"], data["target"]
id2token, token2id = data["id2token"], data["token2id"]

VOCAB_SIZE = len(id2token)
MAX_LEN = data["max_len"]
PAD_ID, START_ID, UNK_ID, END_ID = (token2id[t] for t in SPECIAL_TOKENS)

print("Encoder input :", enc_in.shape)
print("Decoder input :", dec_in.shape)
print("Target        :", target.shape)
print("Example       :", data["sentences"][0])
```

Expected output (reference run):

```
Number of sentences  : 12280
Vocabulary size      : 1898 (1579 rare words mapped to <unk>)
<unk> share of tokens: about 2 %
Max sequence length  : 8
Encoder input : (12280, 8)
Decoder input : (12280, 8)
Target        : (12280, 8)
Example       : ['go', '.', '<end>']
```

The vocabulary size can differ slightly with the NLTK version. All three arrays have the shape (number of sentences, maximum sequence length) and contain integer ids.

6. The Reparameterization Trick

Sampling z directly from $N(\mu, \sigma^2)$ is not differentiable, so gradients cannot flow back to the encoder. The reparameterization trick moves the randomness into a separate noise variable ε:

$$z = \mu + \sigma \odot \varepsilon, \qquad \sigma = e^{\frac{1}{2} \log \sigma^2}, \qquad \varepsilon \sim N(0, I)$$

μ and log σ² are produced by the encoder; ε is random noise. The encoder predicts **log σ²** instead of σ because it can take any real value, whereas σ must be positive.

```
class Sampling(Layer):
    # Reparameterization trick: z = mu + sigma * eps
    def __init__(self, **kwargs):
```

```
        super().__init__(**kwargs)
        self.seed_generator = keras.random.SeedGenerator(SEED)

    def call(self, inputs):
        z_mean, z_log_var = inputs
        eps = keras.random.normal(ops.shape(z_mean), seed=self.seed_generat
or)
        return z_mean + ops.exp(0.5 * z_log_var) * eps


class RepeatToLength(Layer):
    # Copies z to every timestep of the decoder input sequence.
    def call(self, inputs):
        z, ids = inputs
        return tf.tile(tf.expand_dims(z, 1), [1, tf.shape(ids)[1], 1])
```

Details:

- `Sampling` implements the equation above. A `SeedGenerator` makes the noise reproducible.
- `RepeatToLength` copies z to every timestep of the decoder input, so the decoder sees z at every step and not only at the start

7. Constructing the Seq2Seq LSTM-VAE Architecture

The model consists of three Keras models that share layers:

Component	Input → Output	Purpose
Encoder	token ids → μ, log σ^2	Embedding → bidirectional LSTM → two dense heads
z → state	z → (h_0, c_0)	Sets the initial hidden and cell state of the decoder LSTM
Decoder	previous words, z, (h, c) → word logits, new (h, c)	At every step the LSTM reads [embedding of previous word ; z]; a dense layer gives a score for every vocabulary word

```
def build_sentence_vae(vocab_size, embed_dim, hidden_dim, latent_dim):
    embedding = Embedding(vocab_size, embed_dim, mask_zero=True, name="embe
dding")

    # -------- Encoder --------
    enc_ids = Input(shape=(None,), dtype="int32", name="encoder_ids")
    h = Bidirectional(LSTM(hidden_dim), name="encoder_bilstm")(embedding(en
c_ids))
    z_mean = Dense(latent_dim, name="z_mean")(h)
    z_log_var = Dense(latent_dim, name="z_log_var")(h)
    encoder = Model(enc_ids, [z_mean, z_log_var], name="encoder")

    # -------- z -> initial decoder state --------
    z_in = Input(shape=(latent_dim,), name="z")
    h0 = Dense(hidden_dim, activation="tanh", name="z_to_h")(z_in)
    c0 = Dense(hidden_dim, name="z_to_c")(z_in)
    init_state = Model(z_in, [h0, c0], name="z_to_state")

    # -------- Decoder (works for full sequences and single steps) --------
    dec_ids = Input(shape=(None,), dtype="int32", name="decoder_ids")
```

```
    dec_z = Input(shape=(latent_dim,), name="decoder_z")
    h_in = Input(shape=(hidden_dim,), name="h_in")
    c_in = Input(shape=(hidden_dim,), name="c_in")

    z_seq = RepeatToLength(name="repeat_z")([dec_z, dec_ids])
    dec_x = Concatenate(name="word_plus_z")([embedding(dec_ids), z_seq])
    seq, h_out, c_out = LSTM(hidden_dim, return_sequences=True, return_stat
e=True,
                             name="decoder_lstm")(dec_x, initial_state=[h_i
n, c_in])
    logits = Dense(vocab_size, name="vocab_logits")(seq)
    decoder = Model([dec_ids, dec_z, h_in, c_in], [logits, h_out, c_out], n
ame="decoder")

    return encoder, init_state, decoder
```

Details:

- **Shared embedding**: encoder and decoder use the same word vectors
- **Bidirectional encoder**: reads the sentence forwards and backwards, so the summary contains information from both ends
- **One decoder for training and generation**
  - The decoder takes the LSTM state as input and returns the new state
  - During training it processes the whole sequence at once; during generation it is called one word at a time
- **Logits instead of softmax**: the last dense layer has no activation; the softmax is applied inside the loss (`from_logits=True`), which is numerically more stable

8. The Training Objective

The VAE maximises the evidence lower bound (ELBO). In practice we minimise its negative, which has two parts.

Reconstruction loss

The negative log-likelihood of the correct words, **summed** over the real tokens of a sentence ($m_t = 0$ for padding) and averaged over the batch:

$$L_{\text{rec}} = -\sum_t m_t \, log \, p(w_t \mid w_{<t}, z)$$

KL divergence

The distance between the encoder distribution and the prior $N(0, I)$, summed over the latent dimensions d:

$$KL = -\frac{1}{2}\sum_d (1 + log \, {\sigma_d}^2 - {\mu_d}^2 - {\sigma_d}^2)$$

Total loss with KL annealing and free bits

$$L = L_{\text{rec}} + \beta \cdot \sum_d max(KL_d \, , \lambda)$$

- **KL annealing (β)**
  - β = 0 for the first `KL_WARMUP_EPOCHS`, then it rises linearly to `BETA_MAX` over `KL_ANNEAL_EPOCHS`

    - The encoder first learns to put useful information into z, and only then is it pushed towards the prior
- **Free bits** (λ = `FREE_BITS`)
    - KL below λ nats per dimension (averaged over the batch) is not penalised
    - Imposes a minimum KL contribution.
    - This guarantees that z keeps at least about 32 × 0.25 = 8 nats of information about each sentence
- **Word dropout**
    - Before each training step, 25% of the real words in the decoder input are replaced by `<unk>`
    - The decoder can no longer rely only on the previous word and has to use *z*

Because the loss is weighted by β and uses free bits, it is written in a custom `train_step` instead of the standard `model.compile(loss=...)`:

```
class SentenceVAE(keras.Model):
    # Training step: masked reconstruction + annealed, free-bits KL + word
dropout.

    def __init__(self, encoder, init_state, decoder, steps_per_epoch, **kwa
rgs):
        super().__init__(**kwargs)
        self.encoder = encoder
        self.init_state = init_state
        self.decoder = decoder
        self.sampling = Sampling()
        self.warmup_steps = float(KL_WARMUP_EPOCHS * steps_per_epoch)
        self.anneal_steps = float(max(1, KL_ANNEAL_EPOCHS * steps_per_epoch
))
        self.trackers = {k: keras.metrics.Mean(name=k)
                         for k in ["loss", "recon", "kl", "beta", "acc"]}

    @property
    def metrics(self):
        return list(self.trackers.values())

    def call(self, inputs, training=False):
        enc, dec = inputs
        z_mean, _ = self.encoder(enc, training=training)
        h0, c0 = self.init_state(z_mean)
        logits, _, _ = self.decoder([dec, z_mean, h0, c0], training=trainin
g)
        return logits

    def kl_beta(self):
        step = ops.cast(self.optimizer.iterations, "float32")
        return BETA_MAX * ops.clip((step - self.warmup_steps) / self.anneal
_steps, 0.0, 1.0)

    def train_step(self, data):
        x_in, tgt = data[0], data[1]
        enc, dec = x_in[0], x_in[1]
```

```
        # Word dropout: replace real words (id > END_ID) in the decoder inp
ut by <unk>
        drop = tf.random.uniform(tf.shape(dec)) < WORD_DROPOUT
        dec_dropped = tf.where(drop & (dec > END_ID), tf.cast(UNK_ID, dec.d
type), dec)

        with tf.GradientTape() as tape:
            z_mean, z_log_var = self.encoder(enc, training=True)
            z = self.sampling([z_mean, z_log_var])
            h0, c0 = self.init_state(z, training=True)
            logits, _, _ = self.decoder([dec_dropped, z, h0, c0], training=
True)

            mask = ops.cast(tgt > 0, "float32")
            ce = keras.losses.sparse_categorical_crossentropy(tgt, logits,
from_logits=True)
            recon = ops.mean(ops.sum(ce * mask, axis=1))            # nats
per sentence

            kl_dim = -0.5 * (1 + z_log_var - ops.square(z_mean) - ops.exp(z
_log_var))
            kl = ops.mean(ops.sum(kl_dim, axis=1))                  # true
KL, for logging
            kl_fb = ops.sum(ops.maximum(ops.mean(kl_dim, axis=0), FREE_BITS
))

            beta = self.kl_beta()
            loss = recon + beta * kl_fb

        grads = tape.gradient(loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

        pred = ops.cast(ops.argmax(logits, axis=-1), tgt.dtype)
        acc = ops.sum(ops.cast(pred == tgt, "float32") * mask) / ops.sum(ma
sk)

        for k, v in dict(loss=loss, recon=recon, kl=kl, beta=beta, acc=acc)
.items():
            self.trackers[k].update_state(v)
        return {k: t.result() for k, t in self.trackers.items()}
```

Details:

- `train_step` applies word dropout, encodes, samples z, decodes, computes the masked loss, and updates the weights with `tf.GradientTape`
- The logged `kl` is the **true** KL (without free bits), so it shows how much information z really carries
- `acc` is the teacher-forced token accuracy over real tokens only
- `call()` is used by `vae.predict` and decodes from the mean μ (no sampling)

9. Model Training with Teacher Forcing

```
steps_per_epoch = int(np.ceil(len(enc_in) / BATCH_SIZE))

encoder, init_state, decoder = build_sentence_vae(VOCAB_SIZE, EMBED_DIM, HI
DDEN_DIM, LATENT_DIM)
vae = SentenceVAE(encoder, init_state, decoder, steps_per_epoch)
vae.compile(optimizer=keras.optimizers.Adam(learning_rate=LEARNING_RATE, cl
ipnorm=1.0))

encoder.summary()
decoder.summary()

history = vae.fit(
    [enc_in, dec_in], target,
    batch_size=BATCH_SIZE,
    epochs=EPOCHS,
    shuffle=True,
    verbose=2,
)
```

How to read the training log:

- **recon**: reconstruction loss in nats per sentence; it should fall steadily
- **kl**: KL divergence in nats per sentence. It is very large while β = 0, drops quickly once β starts to rise, and must then **stay clearly above 0**. A value close to 0 means posterior collapse.
- **beta**: the current KL weight
- **acc**: teacher-forced token accuracy (padding excluded)
- **loss** (recon + β × KL) rises while β increases; this is expected and not a sign of divergence

Example output (selected epochs from a reference run):

```
Epoch 1/40   acc: 0.3831 - beta: 0.0000 - kl: 721.61 - loss: 21.38 - recon:
21.38
Epoch 2/40   acc: 0.4939 - beta: 0.0000 - kl: 519.64 - loss: 15.99 - recon:
15.99
Epoch 3/40   acc: 0.5368 - beta: 0.0497 - kl:  66.84 - loss: 15.59 - recon:
14.31
Epoch 4/40   acc: 0.5682 - beta: 0.1497 - kl:  12.05 - loss: 14.96 - recon:
13.09
Epoch 13/40  acc: 0.6261 - beta: 1.0000 - kl:   8.19 - loss: 18.70 - recon:
9.88
Epoch 40/40  acc: 0.6930 - beta: 1.0000 - kl:   8.43 - loss: 15.71 - recon:
7.00
```

The KL settles at about 8 nats, just above the free-bits floor of 32 × 0.25 nats: the model keeps z informative. The token accuracy in the log is measured **with** word dropout, so it is lower than the accuracy measured later without dropout.

10. Autoregressive Inference and Decoding Logic

After training there is no teacher. The decoder must generate a sentence on its own (**autoregressive decoding**):

- Start with the start token `\t` and the initial state computed from z
- Predict a score for every word, choose one word, and feed it back as the next input

- Repeat until <end> is produced or the maximum number of **words** is reached

```
def ids_to_text(ids):
    # Turn a sequence of ids into a readable sentence (stops at <end>).
    words = []
    for i in ids:
        tok = id2token[int(i)]
        if tok == END or tok == PAD:
            break
        if tok == START:
            continue
        words.append(tok)
    return " ".join(words)


def decode_latent(Z, max_words=MAX_LEN + 2, temperature=0.0, allow_unk=True
, rng=None):
    # Autoregressively decode a batch of latent vectors Z (B, latent_dim) i
nto id lists.
    # allow_unk=False forbids <unk>, which gives nicer text when generating
from the prior.
    rng = rng or np.random.default_rng()
    Z = np.asarray(Z, dtype="float32").reshape(-1, LATENT_DIM)
    B = Z.shape[0]
    Z_t = tf.convert_to_tensor(Z)                       # Keras 3: don't mix
numpy and tensors
    h, c = init_state(Z_t, training=False)
    tok = tf.fill((B, 1), START_ID)
    out = np.zeros((B, max_words), dtype="int32")
    finished = np.zeros(B, dtype=bool)

    for t in range(max_words):
        logits, h, c = decoder([tok, Z_t, h, c], training=False)
        logits = ops.convert_to_numpy(logits)[:, -1, :].astype("float64")
        logits[:, [PAD_ID, START_ID]] = -1e9                    # never emit
these
        if not allow_unk:
            logits[:, UNK_ID] = -1e9
        if temperature and temperature > 0:
            p = np.exp((logits - logits.max(1, keepdims=True)) / temperatur
e)
            p /= p.sum(1, keepdims=True)
            nxt = np.array([rng.choice(VOCAB_SIZE, p=row) for row in p])
        else:
            nxt = logits.argmax(1)
        nxt[finished] = PAD_ID
        out[:, t] = nxt
        finished |= nxt == END_ID
        if finished.all():
            break
        tok = tf.convert_to_tensor(nxt.reshape(B, 1).astype("int32"))
    return out
```

```
def decode(z, temperature=0.0):
    # Decode a single latent vector to a sentence string.
    return ids_to_text(decode_latent(z, temperature=temperature)[0])


def encode(idx, stochastic=False):
    # Encode sentences (indices into the dataset); returns z (mean or a sam
ple).
    m, lv = encoder.predict(enc_in[np.atleast_1d(idx)], verbose=0)
    if stochastic:
        m = m + np.exp(0.5 * lv) * np.random.randn(*m.shape)
    return m


reference = [ids_to_text(t) for t in target]
```

Details:

- **Greedy decoding** (`temperature=0`): always choose the most likely word. This is deterministic and gives the most fluent output.
- **Temperature sampling** (`temperature > 0`): sample the next word from the softmax of the scores divided by T:

$$p_i = \frac{exp(l_i/T)}{\sum_j exp(l_j/T)}$$

- A low T (for example 0.5) keeps the model close to its most likely choices; a high T (1.0 or more) gives more varied and more surprising sentences, with more mistakes
- The decoder is never allowed to produce `<pad>` or `\t`; with `allow_unk=False` it cannot produce `<unk>` either, which gives nicer generated text
- All sentences in a batch are decoded in parallel, which is much faster than decoding one sentence at a time
- `encode()` returns the mean μ by default, or a random sample $z \sim q(z|x)$ with `stochastic=True`

11. Reconstruction Check

Each sentence is encoded to its mean μ and decoded greedily. `OK` marks an exact reconstruction.

```
idx = np.random.choice(len(enc_in), 10, replace=False)
recon_ids = decode_latent(encode(idx))
for i, r in zip(idx, recon_ids):
    hyp = ids_to_text(r)
    print(f"{'OK ' if hyp == reference[i] else '   '} {reference[i]:<28} ->
{hyp}")
```

Example output:

```
    this is urgent .             -> this is weird .
OK  it 's <unk> .                -> it 's <unk> .
    he 's a dj .                 -> he 's a biologist .
OK  you lied to me .             -> you lied to me .
OK  who will help me ?           -> who will help me ?
    i drink to relax .           -> i want to drink .
    let 's take a taxi .         -> let 's take a coin .
```

```
OK  sit down , tom !              -> sit down , tom !
    he gave me a cold .           -> he gave me a hint .
```

Many sentences are reconstructed exactly. The others are **near-misses**: the structure is kept and one word is replaced by a word that fits the same position. This is typical for a VAE: z stores *what kind of sentence* it is very precisely, but with only about 8 nats it cannot always store the exact word.

12. Latent Space Interpolation (Semantic Exploration)

We encode two sentences A and B, take the points on the straight line between their latent means and decode each point:

$$z(\alpha) = (1-\alpha)\cdot\mu_A + \alpha\cdot\mu_B, \qquad \alpha \in [0,1]$$

```python
for _ in range(NUM_SAMPLES_TO_GENERATE):
    id_from, id_to = np.random.choice(len(enc_in), 2, replace=False)
    z_from, z_to = encode(id_from)[0], encode(id_to)[0]
    alphas = np.linspace(0, 1, 7)
    path = np.stack([(1 - a) * z_from + a * z_to for a in alphas])

    print("==\t", reference[id_from], "==")
    for a, ids in zip(alphas, decode_latent(path)):
        print(f"{1 - a:.2f}\t", ids_to_text(ids))
    print("==\t", reference[id_to], "==\n")
```

Example output:

```
==   stop screaming . ==
1.00     stop yelling .
0.83     stop yelling .
0.67     i 'm <unk> .
0.50     i feel lonely .
0.33     i feel good .
0.17     i feel good .
0.00     i feel good .
==   i feel normal . ==
```

What you should observe:

- The end points reproduce (or closely resemble) the two input sentences
- Every intermediate point decodes to a **grammatical sentence**, not a mixture of words
- Sentences change gradually: neighbouring points in the latent space correspond to similar sentences

We interpolate between the means μ instead of random samples, because samples add noise and make the path less clear.

13. Generating New Sentences from the Prior

Since the KL term pulls the latent space towards $N(0, I)$, we can sample $z \sim N(0, I)$ and decode it into a sentence that was never given to the model. Each output is marked `seen` (it exists in the training data) or `NEW`.

```python
train_set = set(reference)
Z_prior = np.random.randn(15, LATENT_DIM)
for ids in decode_latent(Z_prior, allow_unk=False):
    s = ids_to_text(ids)
```

```
    print(f"{'seen' if s in train_set else 'NEW':>4}  {s}")

print("\nWith temperature 0.7 (more variety):")
for ids in decode_latent(Z_prior, temperature=0.7, allow_unk=False):
    s = ids_to_text(ids)
    print(f"{'seen' if s in train_set else 'NEW':>4}  {s}")
```

Example output (excerpt):

```
seen  no one loves war .
 NEW  that 's new .
 NEW  i got a lot .
 NEW  i 'll take this .
seen  i need a bath .

With temperature 0.7 (more variety):
 NEW  the bill is home .
 NEW  i 'm a bit sick .
 NEW  i 'll check my way .
seen  we must obey .
```

14. Novelty and the Temperature Trade-off

To measure how generative the model is, we decode 500 random latent vectors at several temperatures and compute:

- **Novel**: the share of samples that are not sentences from the training set
- **Distinct**: the share of samples that are different from each other

```
Z_eval = np.random.randn(500, LATENT_DIM)
rng = np.random.default_rng(SEED)
for temp in [0.0, 0.5, 0.8, 1.0]:
    samples = [ids_to_text(r) for r in decode_latent(Z_eval, temperature=te
mp,
                                                     allow_unk=False, rng=r
ng)]
    novel = np.mean([s not in train_set for s in samples])
    distinct = len(set(samples)) / len(samples)
    new_examples = [s for s in dict.fromkeys(samples) if s not in train_set
][:4]
    print(f"temperature {temp:.1f}:  novel {100 * novel:5.1f} %   distinct
{100 * distinct:5.1f} %")
    for s in new_examples:
        print("      NEW ", s)
    print()
```

Example results (reference run):

Temperature	Novel	Distinct	Examples of new sentences
0.0 (greedy)	36.0%	90.0%	i got in charge . / that 's time .
0.5	48.0%	98.6%	what 's the party ? / i got at school .
0.8	62.0%	99.0%	where can we eat ? / let me continue .
1.0	73.2%	99.0%	did you eat pork ? / what month is all ?

Higher temperatures give more novel and more varied sentences, but grammar and meaning gradually get worse ("what month is all ?"). The numbers alone do not show this, so always **read the examples** as well.

15. Quantitative Evaluation: BLEU Score and Edit Distance

In Exercise 1 we used the Mean Squared Error. For text, MSE is not meaningful: words are discrete, and there is no "average" between two words. Instead, we compare the decoded sentence (hypothesis) with the original (reference):

- **BLEU-2** (Bilingual Evaluation Understudy) counts how many single words and word pairs of the hypothesis also appear in the reference (0 = no overlap, 1 = identical)
    - **Word BLEU-2** compares words; **character BLEU-2** compares characters and gives partial credit for similar words
- **Exact match**: the share of sentences reconstructed without any error
- **Normalised edit distance**: the minimum number of word insertions, deletions and substitutions needed to turn the hypothesis into the reference, divided by the reference length (0 is perfect)

$$Normalised\ distance = \frac{EditDistance(ref, hyp)}{|ref|}$$

Both are computed for **deterministic** reconstruction ($z = \mu$) and **stochastic** reconstruction (z sampled from $q(z|x)$).

```
smooth = SmoothingFunction().method1
BLEU2 = (0.5, 0.5, 0.0, 0.0)


def edit_distance(a, b):
    # Levenshtein distance between two token lists.
    prev = list(range(len(b) + 1))
    for i, ta in enumerate(a, 1):
        cur = [i]
        for j, tb in enumerate(b, 1):
            cur.append(min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + (ta !
= tb)))
        prev = cur
    return prev[-1]


def score_pair(ref, hyp):
    r, h = ref.split(), hyp.split()
    word_bleu = sentence_bleu([r], h, weights=BLEU2, smoothing_function=smo
oth) if h else 0.0
    char_bleu = (sentence_bleu([list(ref)], list(hyp), weights=BLEU2, smoot
hing_function=smooth)
                 if len(hyp) > 1 else 0.0)
    ned = edit_distance(r, h) / max(len(r), 1)
    return word_bleu, char_bleu, float(ref == hyp), ned


def evaluate_reconstruction(num_samples=500, stochastic=False, seed=SEED):
    rng = np.random.default_rng(seed)
    idx = rng.choice(len(enc_in), min(num_samples, len(enc_in)), replace=Fa
lse)
```

```
    hyps = [ids_to_text(r) for r in decode_latent(encode(idx, stochastic=st
ochastic))]
    scores = np.array([score_pair(reference[i], h) for i, h in zip(idx, hyp
s)])
    return dict(zip(["word BLEU-2", "char BLEU-2", "exact match", "norm. ed
it dist"],
                    scores.mean(0)))


for name, stoch in [("Deterministic (z = mu)", False), ("Stochastic (z ~ q(
z|x))", True)]:
    res = evaluate_reconstruction(stochastic=stoch)
    print(f"{name:<26}" + "  ".join(f"{k}: {v:.3f}" for k, v in res.items()
))
```

Example results (reference run, 500 sentences):

Reconstruction	Word BLEU-2	Char BLEU-2	Exact match	Norm. edit distance
Deterministic (z = μ)	0.601	0.711	0.278	0.236
Stochastic (z ~ q(z|x))	0.348	0.534	0.068	0.449

Stochastic reconstruction is clearly worse. The posterior standard deviation is about 0.8 in most dimensions, so a random sample from $q(z|x)$ often lands in the region of a neighbouring sentence.

16. Token Accuracy

Token accuracy measures, with teacher forcing and $z = \mu$, how often the decoder predicts the correct next word. Only real tokens are counted, not padding.

```
correct, total = 0, 0
for s in range(0, len(enc_in), 1024):            # in chunks: full logits
would need ~1 GB
    sl = slice(s, s + 1024)
    pred = vae.predict([enc_in[sl], dec_in[sl]], batch_size=256, verbose=0)
.argmax(-1)
    mask = target[sl] > 0
    correct += (pred == target[sl])[mask].sum()
    total += mask.sum()
print(f"Token accuracy (non-padding): {correct / total:.4f}")
```

Example output:

```
Token accuracy (non-padding): 0.8429
```

Token accuracy (84%) is much higher than the exact-match rate (28%). With teacher forcing the decoder always receives the correct previous word. With free generation one wrong word, changes all following inputs, so errors add up.

17. Convergence Diagnostics and Latent Histograms

The following cells check whether training was healthy and whether the latent space is really used.

```
hist = history.history
fig, ax = plt.subplots(1, 2, figsize=(13, 4))
ax[0].plot(hist["recon"], label="Reconstruction (nats/sentence)")
ax[0].plot(hist["loss"], label="Total loss")
```

```
ax[0].set_xlabel("Epoch"); ax[0].set_title("Training loss"); ax[0].legend()

ax[1].plot(hist["kl"], color="tab:red", label="KL (nats/sentence)")
ax[1].set_yscale("log")   # KL is in the hundreds during warm-up and ~8 later; a linear axis would hide the final value
ax[1].set_xlabel("Epoch"); ax[1].set_ylabel("KL (log scale)"); ax[1].set_title("KL divergence and KL weight")
ax2 = ax[1].twinx()
ax2.plot(hist["beta"], color="gray", ls="--", label="beta")
ax2.set_ylabel("beta")
ax[1].legend(loc="upper left"); ax2.legend(loc="upper right")
plt.tight_layout(); plt.show()
```

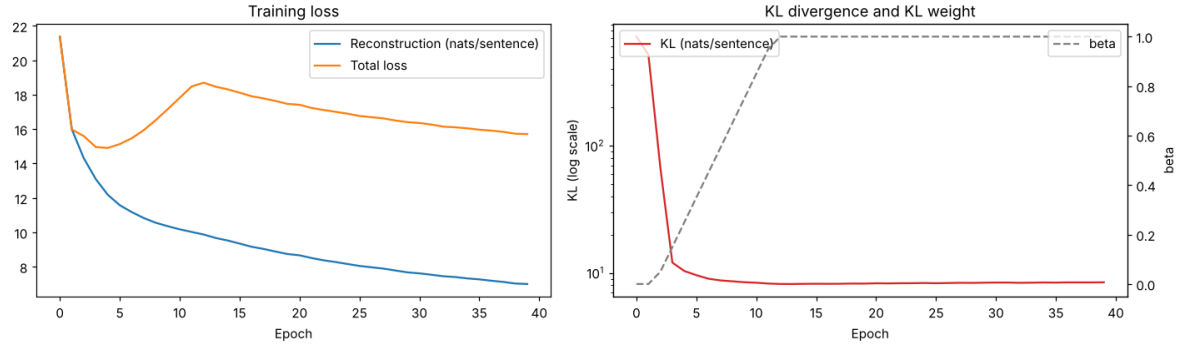


Figure 1: Loss curves (left) and KL divergence on a logarithmic axis with the KL weight β (right) from the reference run.

```
z_mean_all, z_log_var_all = encoder.predict(enc_in, batch_size=256, verbose=0)
var_of_means = z_mean_all.var(axis=0)
active = var_of_means > 0.01
print(f"Active latent units: {active.sum()} / {LATENT_DIM}")

kl_per_sentence = -0.5 * np.sum(1 + z_log_var_all - z_mean_all**2 - np.exp(z_log_var_all), axis=1)
print(f"Mean KL per sentence: {kl_per_sentence.mean():.2f} nats")

fig, ax = plt.subplots(1, 3, figsize=(15, 4))
ax[0].hist(z_mean_all[:, active].flatten(), bins=50)
ax[0].set_title("Latent means (active dims)")
ax[1].hist(np.exp(0.5 * z_log_var_all).flatten(), bins=50)
ax[1].set_title("Latent std dev (all dims)")
ax[2].bar(range(LATENT_DIM), var_of_means)
ax[2].axhline(0.01, color="red", ls="--")
ax[2].set_title("Var of mu per dimension"); ax[2].set_xlabel("latent dim")
plt.tight_layout(); plt.show()
```

Example output:

```
Active latent units: 32 / 32
Mean KL per sentence: 8.00 nats
```

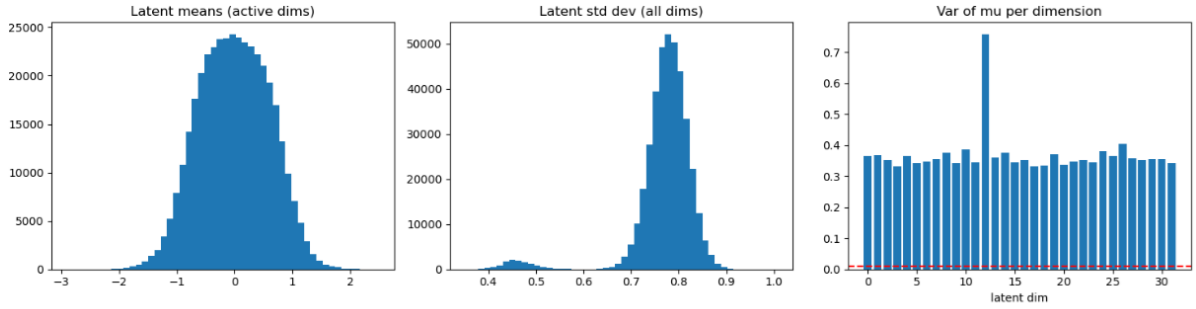


Figure 2: Latent means of all sentences (left), posterior standard deviations (middle) and the variance of μ per latent dimension (right).

Details:

- **The KL axis is logarithmic.** During warm-up (β = 0) the KL is in the hundreds; afterwards it is about 8 nats. On a linear axis the final value would look like 0 and could be mistaken for posterior collapse. Always check the value itself in the training log or in the `Mean KL per sentence` output.
- **Active units**: a latent dimension d is active if its mean changes across sentences:

$$Var_x[\mu_d(x)] > 0.01$$

- With posterior collapse, the number of active units is 0. Here all 32 dimensions are used.
- **Latent means** form a bell curve centred at 0, as the KL term intends
- **Posterior standard deviations** around 0.8 show that each dimension stores only part of the information; a value of 1 would mean an unused dimension
- **Aggregate posterior**: for every dimension, Var[μ] + E[σ²] ≈ 0.35 + 0.61 ≈ 1, so the encoded sentences together cover $N(0, I)$. This is why sampling from the prior gives sensible sentences.
- One dimension (12 in the reference run) has a much larger variance of μ and a smaller σ. It carries more information than the others; the traversal below shows what it encodes.

Summary of healthy behaviour:

Diagnostic	Healthy behaviour	Warning sign
Reconstruction loss	Decreases steadily	Stays flat from the start
KL divergence	Settles clearly above 0 (here about 8 nats)	Drops to about 0 (posterior collapse)
Total loss	Rises while β increases, then decreases	Keeps rising after β = 1
Active units	Most or all dimensions active	0 active units
Latent means	Bell-shaped, centred at 0	All values near 0
Posterior σ	Clearly below 1	All values close to 1

18. Latent Traversal

We start from the encoding of one sentence and change **one** latent dimension at a time, keeping the others fixed. We use the two most active dimensions, because changing an inactive dimension does not change the output.

```
def latent_traversal(z_base, dim, values):
    Z = np.repeat(z_base[None, :], len(values), axis=0)
    Z[:, dim] = values
    for v, ids in zip(values, decode_latent(Z)):
```

```
        print(f"z[{dim}] = {v:+.2f} -> {ids_to_text(ids)}")


z0 = encode(0)[0]
print("Base sentence:", reference[0], "\n")
top_dims = np.argsort(var_of_means)[::-1][:2]
spread = np.sqrt(var_of_means[top_dims[0]])
for d in top_dims:
    latent_traversal(z0, d, z0[d] + spread * np.linspace(-3, 3, 7))
    print()
```

Example output (base sentence "go ."):

```
z[12] = -1.11 -> i like to go .
z[12] = -0.24 -> go to bed .
z[12] = +0.63 -> go away .
z[12] = +1.51 -> go inside .
z[12] = +2.38 -> go .
z[12] = +3.25 -> jump .
```

Along dimension 12 the sentences become **shorter**: the model has learned, without being told, a latent direction that mainly encodes sentence length. Other dimensions encode other properties, and some changes are only visible from other starting sentences.

Conclusion of Exercise 2

In this exercise we built a Seq2Seq Variational Autoencoder for sentences and showed that it learns a continuous latent space for language:

- **The latent space is used**: the KL divergence stays at about 8 nats and all 32 dimensions are active. Without KL annealing, word dropout, free bits and z at every decoder step, a text VAE easily suffers from posterior collapse.
- **Reconstruction**: about 28% of the sentences are reconstructed exactly from μ, and most of the others differ by one similar word (word BLEU-2 ≈ 0.6).
- **Smooth interpolation**: points between two sentences decode to grammatical intermediate sentences that change gradually.
- **Generation**: sampling $z \sim N(0, I)$ produces new sentences. The temperature controls the balance between novelty and fluency.
- **Interpretable directions**: some latent dimensions capture properties such as sentence length without supervision.

Recommendations to further improve the model:

- **Evaluation on held-out data**
  - All metrics here are computed on training sentences. A train/validation split, with the ELBO or perplexity on the validation set, measures generalisation more honestly.
- **Decoding**
  - **Beam search** keeps the k best partial sentences instead of only the best word, which usually gives more fluent output than greedy decoding
  - **Top-k or nucleus (top-p) sampling** avoids very unlikely words while keeping variety
- **Representation**
  - **Subword tokenization** (for example byte-pair encoding) removes most <unk> tokens
  - **Pre-trained embeddings** (for example GloVe) help when the dataset is small
- **Architecture**
  - A **Transformer** encoder and decoder can model longer sentences
  - A **conditional VAE** can control properties such as tense or sentiment with an extra label input

Questions and Experiments

1. **Posterior collapse.** Set `WORD_DROPOUT = 0`, `FREE_BITS = 0`, `KL_WARMUP_EPOCHS = 0` and `KL_ANNEAL_EPOCHS = 0`, then retrain. What happens to the KL divergence, the number of active units and the latent traversals? Explain why.
2. **Amount of information in z.** Retrain with `FREE_BITS = 0.5`. How do the exact-match rate and the novelty rate change? Explain the trade-off between reconstruction and generation.
3. **Dataset size.** Retrain with `NUM_SAMPLES = 3000` and compare the novelty rates with your 20,000-line model. Why does the model trained on less data produce mostly sentences it has already seen? What does this tell you about memorisation and generalisation in generative models?
4. **Deterministic vs. stochastic reconstruction.** Why is reconstruction from a sample $z \sim q(z|x)$ worse than from μ? Use the histogram of posterior standard deviations in your answer.
5. **Temperature.** Which temperature would you choose to generate new but still sensible sentences? Support your choice with examples from your own results.
6. **Interpreting the latent space.** Find the latent dimension with the largest variance of μ in your model and traverse it from several different starting sentences. What property of the sentence does it seem to encode?
7. **Teacher forcing vs. free generation.** Why is the teacher-forced token accuracy much higher than the exact-match rate of greedy decoding?